Al fin(ito) y más allá del bearer token

Un billete al portador llamado token

Seguridad de APIs · de Basic al DPoP

2026 · charla técnica · 28 diapositivas

Un token es un billete al portador: quien lo tiene, entra

- posesión = autorización
- "bearer" viene del derecho mercantil
- no identifica al portador
- RFC 6750 (oct. 2012)

Fuente: RFC 6750, "Bearer Token Usage", octubre 2012.

La norma lo admitió primero: no puedo saber a quién le emití esto

«La especificación no provee ningún método para que el servidor de recursos se asegure de que un access token presentado por un cliente le fue emitido realmente a ese cliente.»

RFC 6749, §10.3 — octubre 2012

Fuente: RFC 6749 §10.3, octubre 2012.

Alrededor del token hay un sistema entero



Fuente: OWASP API Security Top 10 (2023).

No todas las APIs son REST: cada estilo tiene su propia falla

REST	GraphQL	gRPC	SOAP	WebSocket	Webhooks
recursos	*consultas*	*binario*	*XML*	*canal*	*push*
BOLA / IDOR	query depth / sobre-exposición	WAF no inspecciona	XXE	auth solo al inicio	SSRF / replay

Fuente: OWASP API Security Top 10 (2023).

Primero mandábamos la contraseña; después, una cookie

- Basic manda usuario:contraseña en Base64 por request
- Digest mejora el hash, no el modelo
- la cookie guarda estado en el servidor

Fuente: RFC 2617 (1999) / RFC 7616 (2015); RFC 6265 (2011).

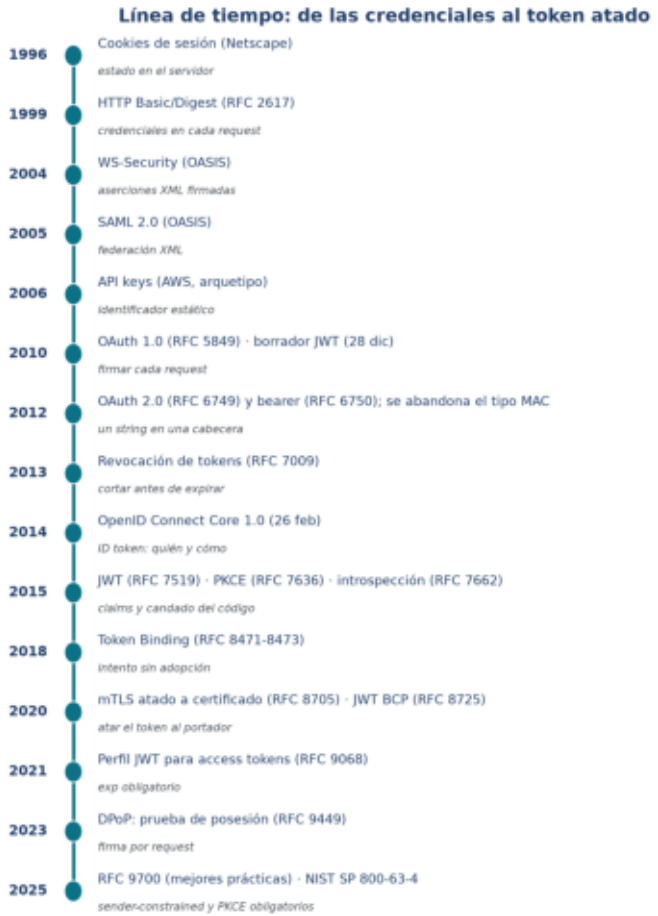
Lo firmado era más seguro; lo simple ganó igual

- SAML 2.0 (OASIS, 2005) y WS-Security (2004) firmaban aserciones XML
- OAuth 1.0 (RFC 5849, 2010) firmaba cada request con HMAC-SHA1
- más seguro, más pesado

Fuente: SAML 2.0 (2005); WS-Security (2004); RFC 5849 (2010).

OAuth 2.0 nació con dos tipos de token; el mundo eligió uno

- RFC 6749 (oct. 2012) definía bearer y mac
- el tipo mac firmaba cada request
- el borrador MAC (feb. 2012) se abandonó
- ganó la simplicidad



Fuente: RFC 6749 §7.1 (2012); borrador MAC, febrero 2012.

No hay 'un token': hay una familia con roles distintos

- authorization code (un solo uso) → access token (corto) → refresh token (largo, rotación) + ID token (identifica, no autoriza)



Fuente: RFC 6749 §4.1 / §5 (2012); OIDC Core, 26 de febrero de 2014.

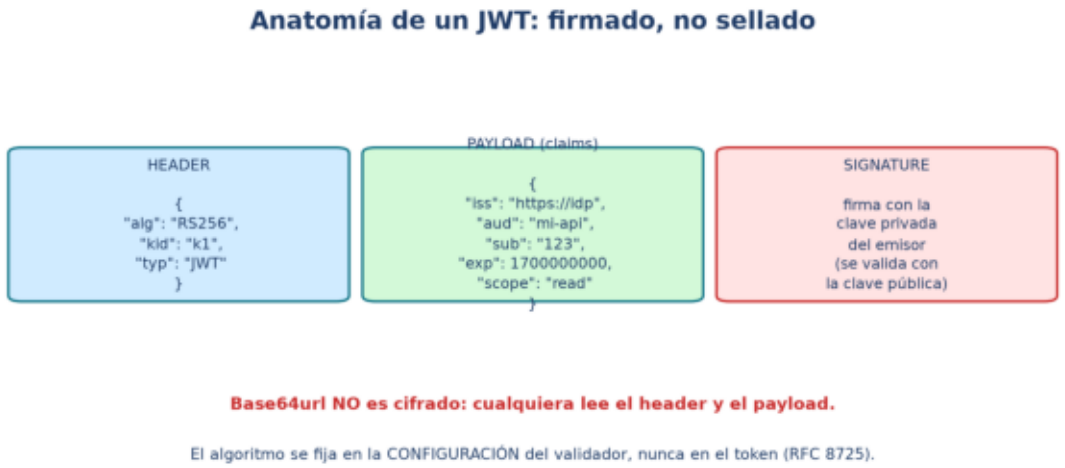
Cada token tiene su destinatario y su vida

- código (front channel, un uso)
- access (la API, corto)
- refresh (el endpoint de token, largo, rota)
- ID (la app, identifica)

Fuente: RFC 6749 §§1.5, 4.1, 5, 6 (2012); RFC 7009 (2013).

Está firmado, no sellado: el payload lo lee cualquiera

- header.payload.signature en base64url
- base64 no es cifrado
- el algoritmo NO se toma del token (RFC 8725)



Fuente: RFC 7519 (2015); RFC 8725 (feb. 2020).

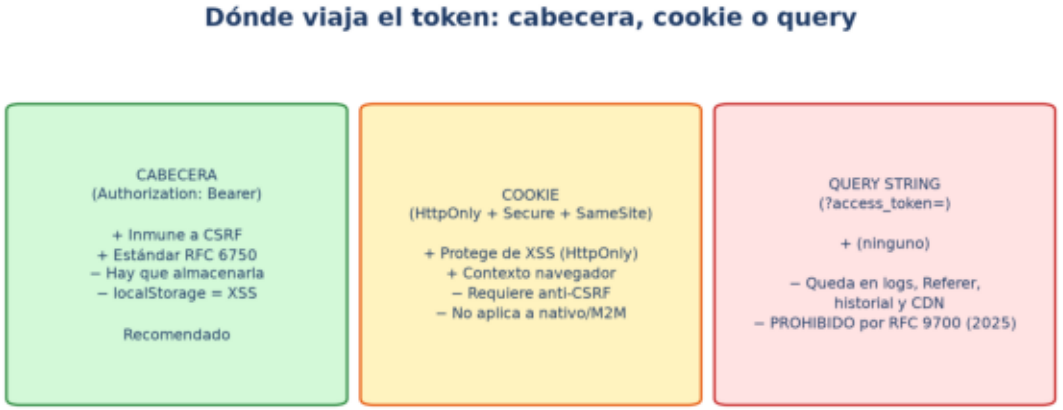
Un JWT bien validado responde cuatro preguntas

- ¿quién lo emitió? (iss)
- ¿es para mí? (aud)
- ¿está vigente? (exp/nbf)
- ¿tiene permiso? (scope)
- amr/acr dicen cómo se autenticó

Fuente: RFC 7519 (2015); RFC 8176 (2017); RFC 9068 (2021).

Trece años tardamos en prohibir el token en la URL

- RFC 6750 permitía cabecera, cuerpo y query
- RFC 9700 (enero 2025) prohíbe tokens en URIs
- cabecera vs cookie: XSS vs CSRF



Fuente: RFC 6750 §2 (2012); RFC 9700 (enero 2025).

La MFA protege un instante; el token vive 30 días

- el flujo real es password → MFA → emisión → token (30 días) → API
- la API no sabe si hubo MFA salvo que se lo digan los claims



Fuente: NIST SP 800-63-4 (jul. 2025); OIDC Core (2014).

Passkeys son el único factor que sobrevive al phishing

- NIST AAL1-AAL3
- WebAuthn/passkeys (W3C, 2019) resisten phishing
- TOTP se phishea en tiempo real
- SMS es el más débil
- step-up con acr/amr

Fuente: NIST SP 800-63-4 (jul. 2025); WebAuthn (2019); RFC 8176 (2017).

La MFA cubre el login, no lo que pasa después

- fatiga de push (MITRE T1621)
- phishing AiTM roba la sesión ya autenticada
- infostealers y pass-the-cookie roban el artefacto

Fuente: MITRE ATT&CK T1621; T1539; casos públicos 2022.

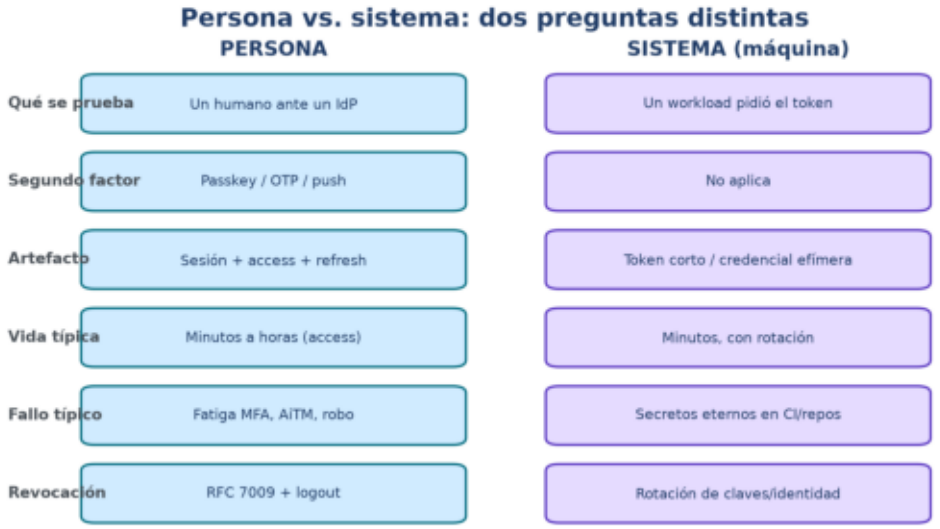
A una máquina no le podés pedir un segundo factor

- el invocador es microservicio, CI o IoT
- no hay usuario
- la pregunta es qué workload habla y con qué credencial de vida corta
- mecanismos: client credentials, private_key_jwt, mTLS, DPoP

Fuente: RFC 6749 §4.4 (2012); RFC 7523 (2015); RFC 8705 (2020).

Dos preguntas distintas, no una sola mal aplicada

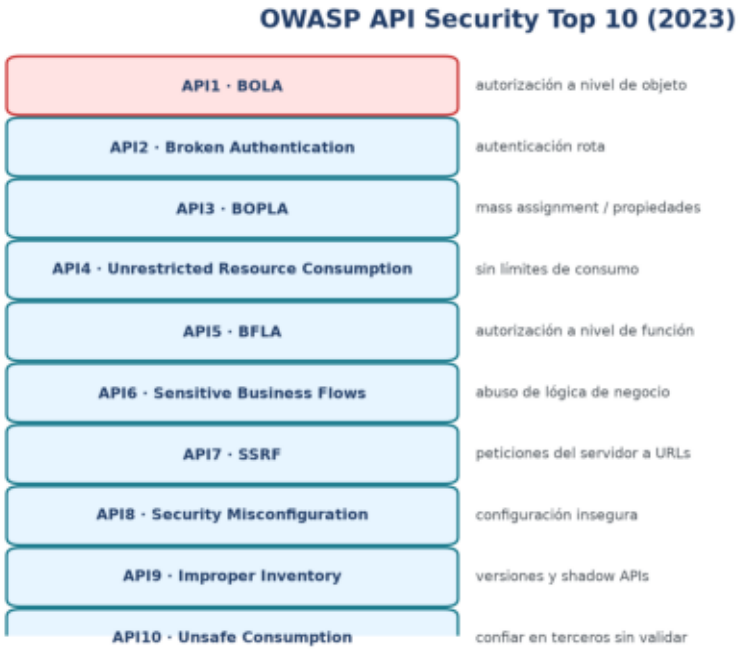
- persona: ¿quién pasó la MFA?
- sistema: ¿qué credencial de vida corta me habla?
- el fallo típico de cada lado



Fuente: NIST SP 800-63-4 (2025); GitGuardian Secrets Sprawl 2026.

El token es una puerta; los riesgos viven en todo el edificio

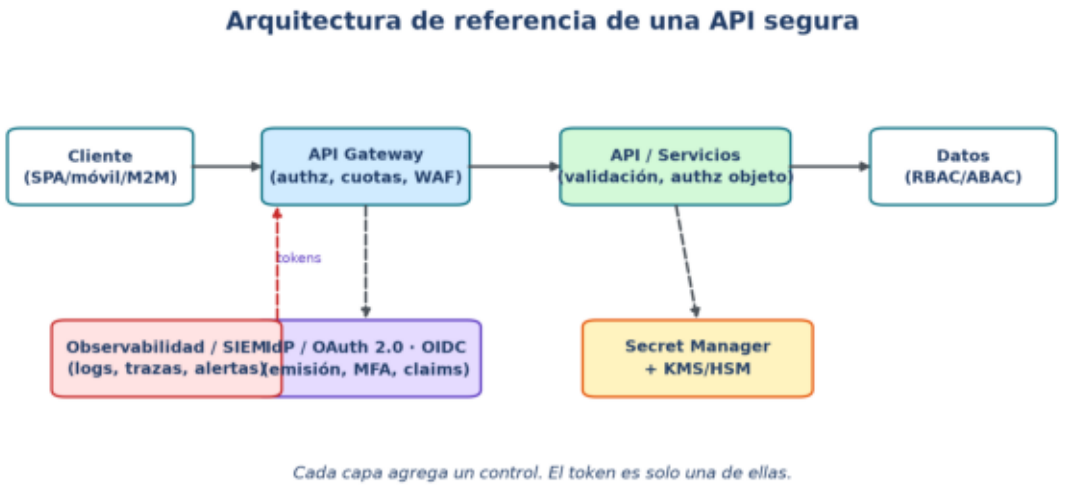
- BOLA (Api1)
- Broken Authentication (Api2)
- BOPLA (Api3)
- consumo y versión (Api4/9)
- negocio y configuración



Fuente: OWASP API Security Top 10, edición 2023.

El token es una puerta; el edificio tiene más habitaciones

- autorización a nivel de objeto
- validación e inyección
- gateway y rate limiting
- inventario y versiones
- cadena de suministro



Cada capa agrega un control. El token es solo una de ellas.

Fuente: OWASP API Top 10 (2023); NIST; AGESIC / CERTuy.

El token funcionó exactamente como estaba diseñado

- Facebook "View As" (2018): token emitido al usuario equivocado
- Okta/Cloudflare (2022): sesión robada y no revocada
- Codecov (2021): tokens de CI filtrados

Fuente: Meta (2018); Cloudflare (2022); Codecov (2021).

Después de aceptarlo, todavía queda trabajo

- Uber (2022): la MFA no sobrevive al robo de sesión
- BOLA: token válido, objeto ajeno (~40 % de ataques a APIs)
- JWT mal validado: alg:none, confusión HS/RS, kid

Fuente: UpGuard/NYT (2022); OWASP 2023; RFC 8725 (2020).

Si la posesión no basta, exijamos prueba de posesión

- mTLS + token atado al certificado (RFC 8705, 2020)
- DPoP firma por request con jti/htm/htu (RFC 9449, 2023)
- RFC 9700 (2025) recomienda sender-constrained

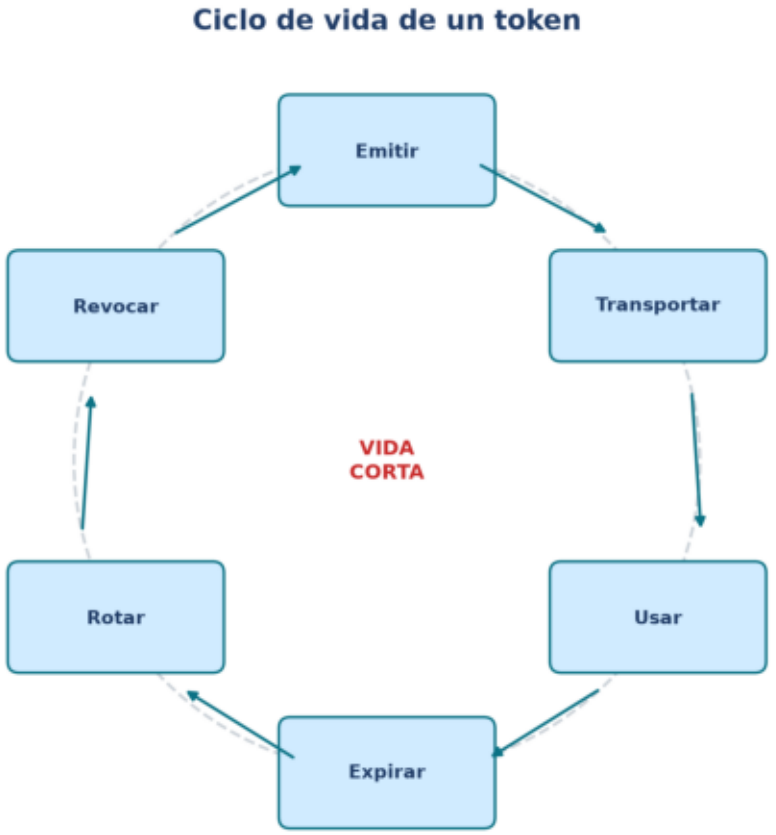


RFC 9700 (2025): los tokens sender-constrained se recomiendan donde sea posible.

Fuente: RFC 8705 (2020); RFC 9449 (2023); RFC 9700 (2025).

La rotación convierte el refresh en un artefacto de un solo uso

- el refresh rotado delata el reuso
- RFC 7009 define la revocación
- introspección (RFC 7662) para tokens opacos
- relación entre vida del JWT y revocación



Fuente: RFC 6749 §10.4 (2012); RFC 7009 (2013); RFC 9700 (2025).

El token se acaba; el riesgo no

- vida corta y aud estricto
- rotación y revocación
- mínimo privilegio
- detección de anomalías
- secret manager para claves de máquina

Fuente: RFC 9700 (2025); RFC 7009 (2013); GitGuardian 2026.

Ocho preguntas antes de dar por segura una API

- ¿el algoritmo viene de la config?
- ¿validás iss/aud/exp?
- ¿la vida es corta?
- ¿hay rotación y revocación?
- ¿autorizás por objeto?

Fuente: RFC 8725 (2020); RFC 9700 (2025); OWASP API Top 10 (2023).

Un dominio, un mensaje: lo que hay que llevarse

- identidad (¿quién o qué?)
- autorización (¿sobre qué objeto?)
- entrada (¿confío? no)
- tráfico (¿límites?)
- observabilidad (¿lo veo?)

Fuente: OWASP API Top 10 (2023); NIST SP 800-63-4 (2025); RFC 9700 (2025).

Al fin(ito) y más allá: el token se acaba, el riesgo no

- (a) el token es finito: emitir, rotar, revocar
- (b) la MFA protege la emisión, no la posesión
- (c) autenticar es la puerta, autorizar es la cerradura

Fuente: bibliografía consolidada al final.

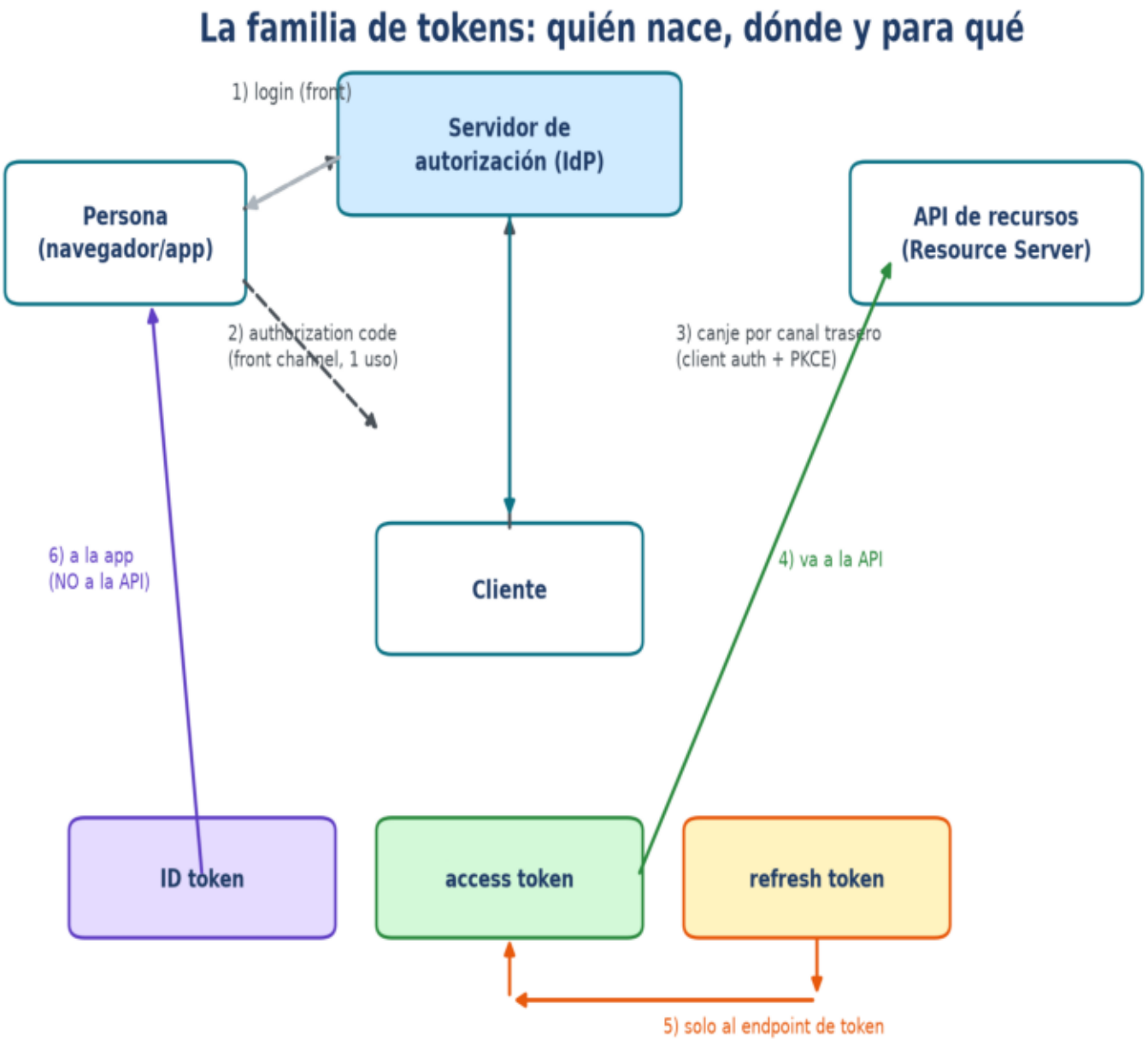
ANEXO — DIAGRAMAS

13 diagramas en página completa

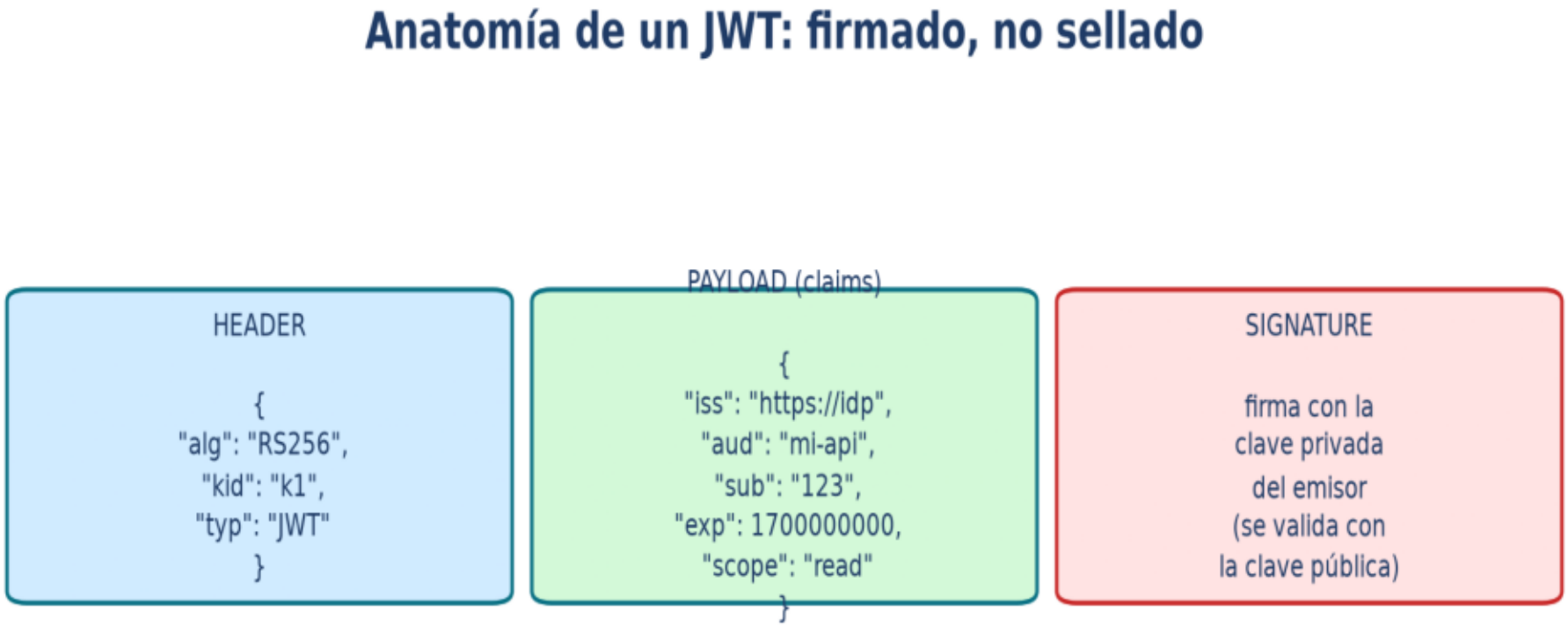
1. Línea de tiempo: de las credenciales al token atado



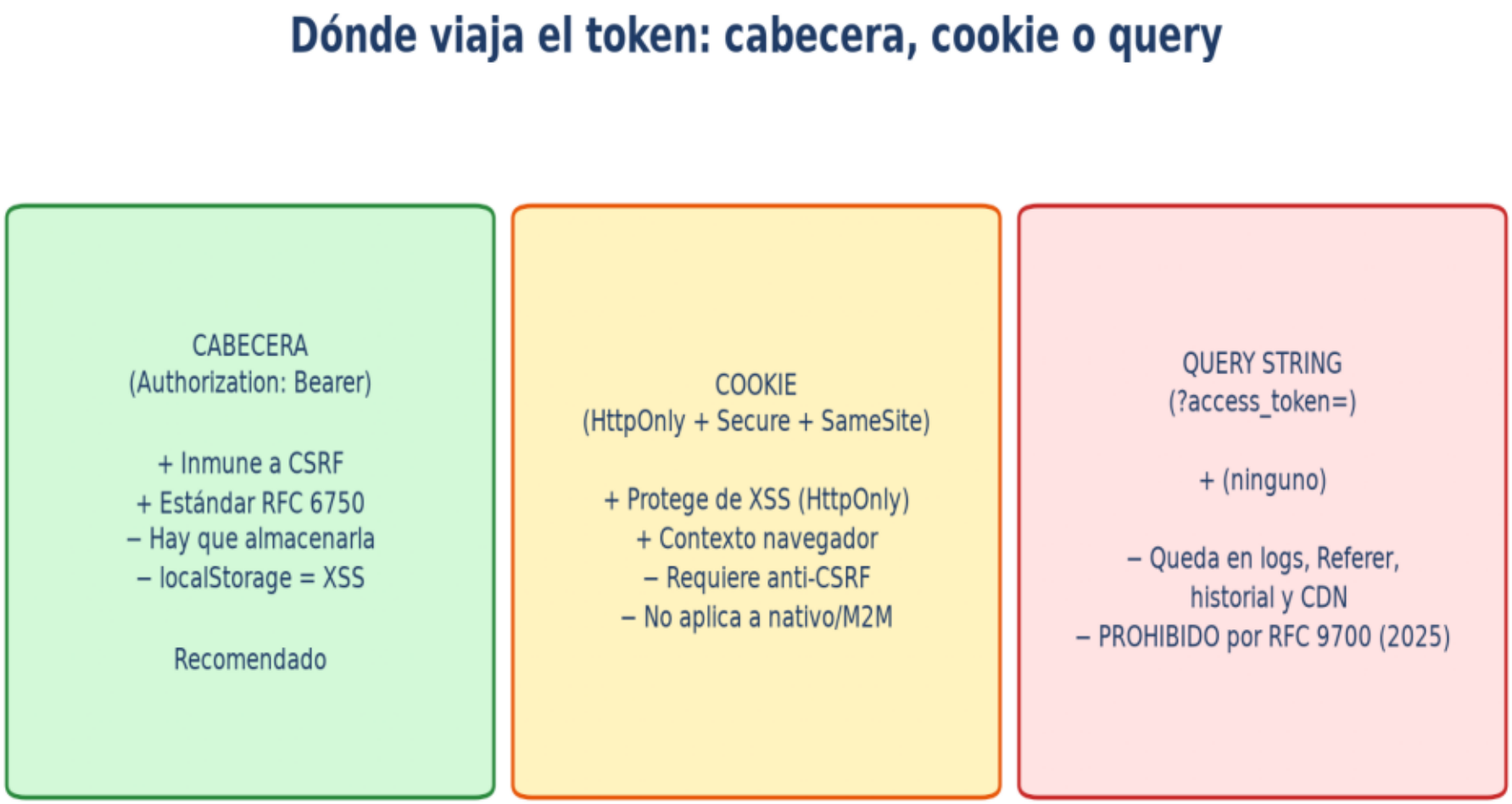
2. La familia de tokens y el flujo de emisión



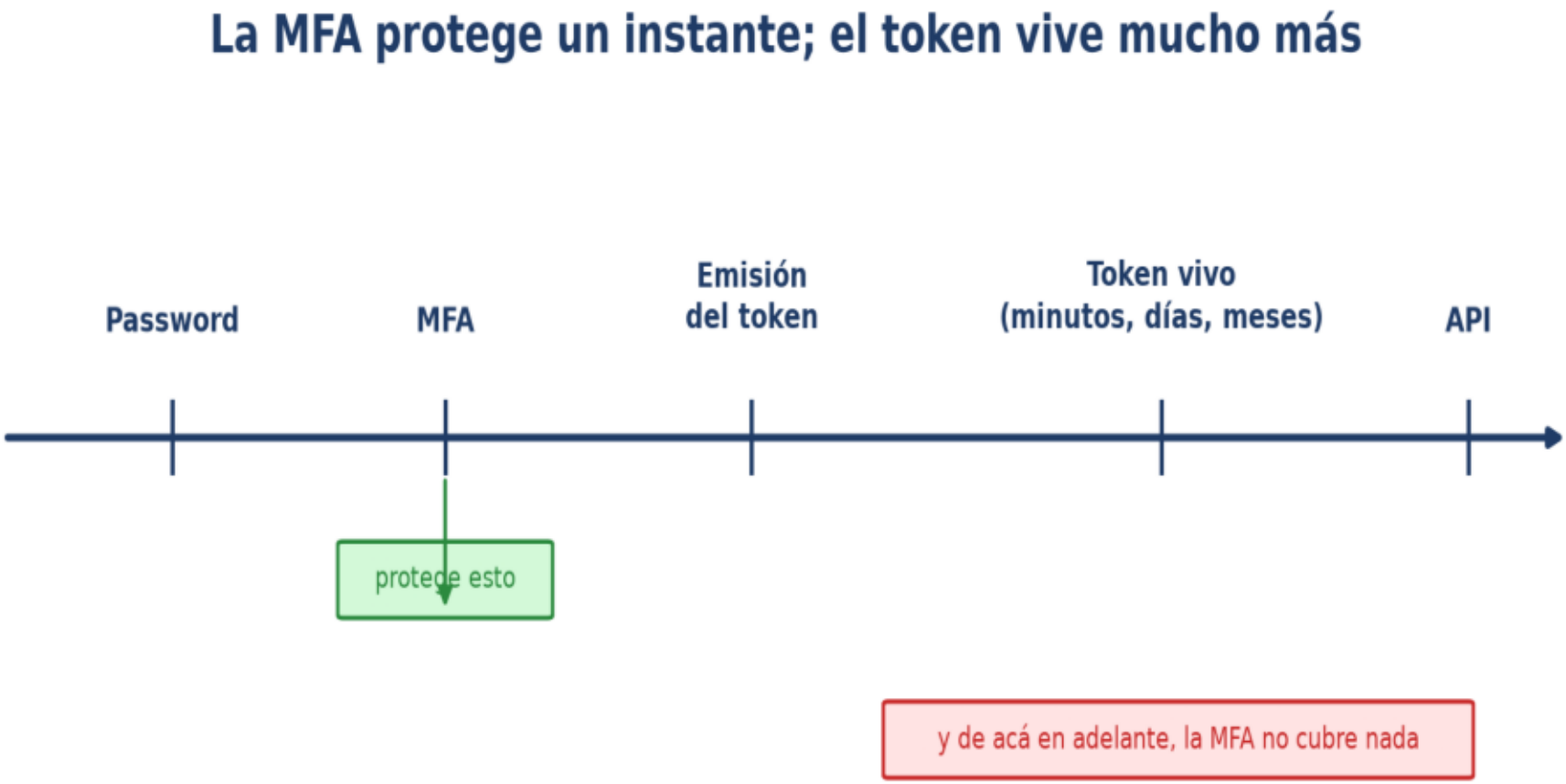
3. Anatomía de un JWT: firmado, no sellado



4. Dónde viaja el token: cabecera, cookie o query



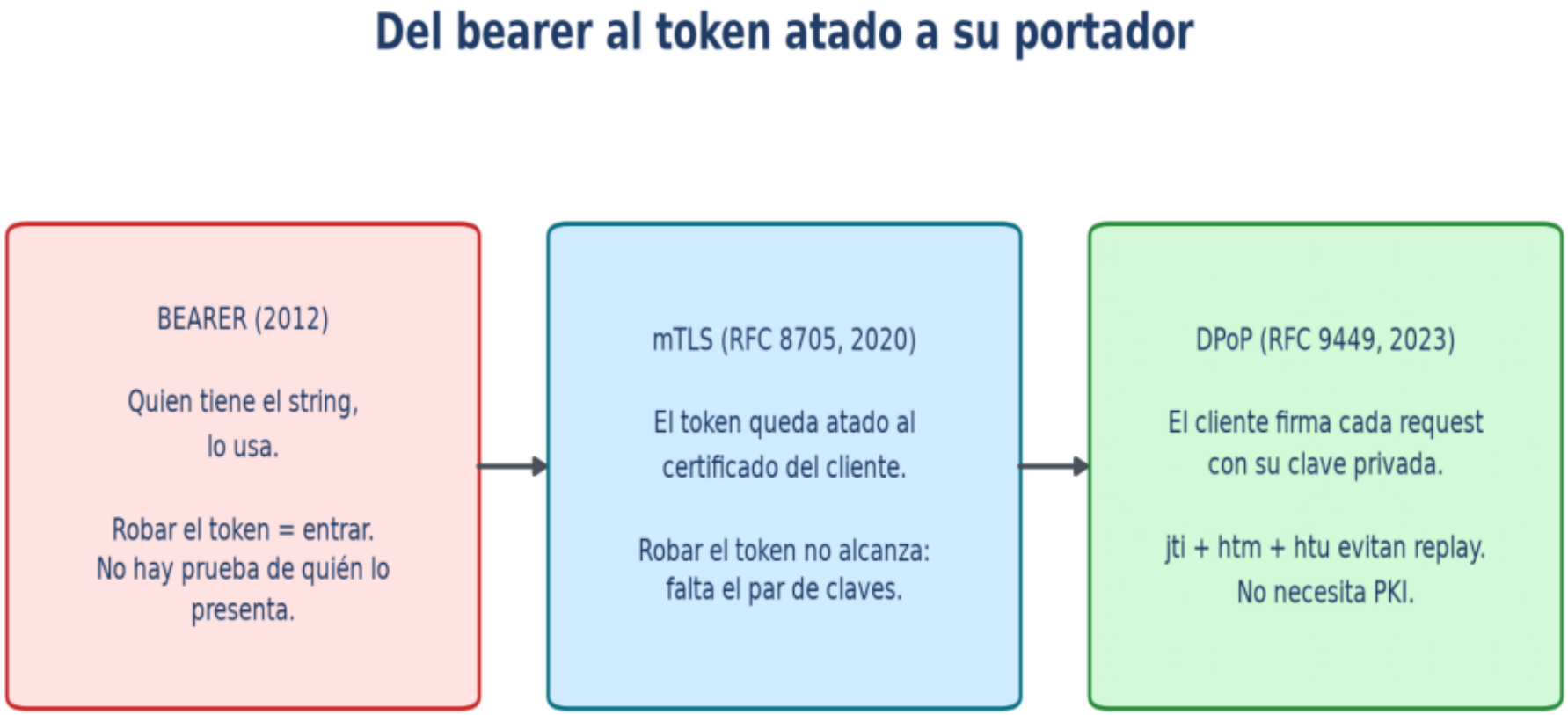
5. La MFA protege un instante; el token vive mucho más



6. Persona vs. sistema: dos preguntas distintas

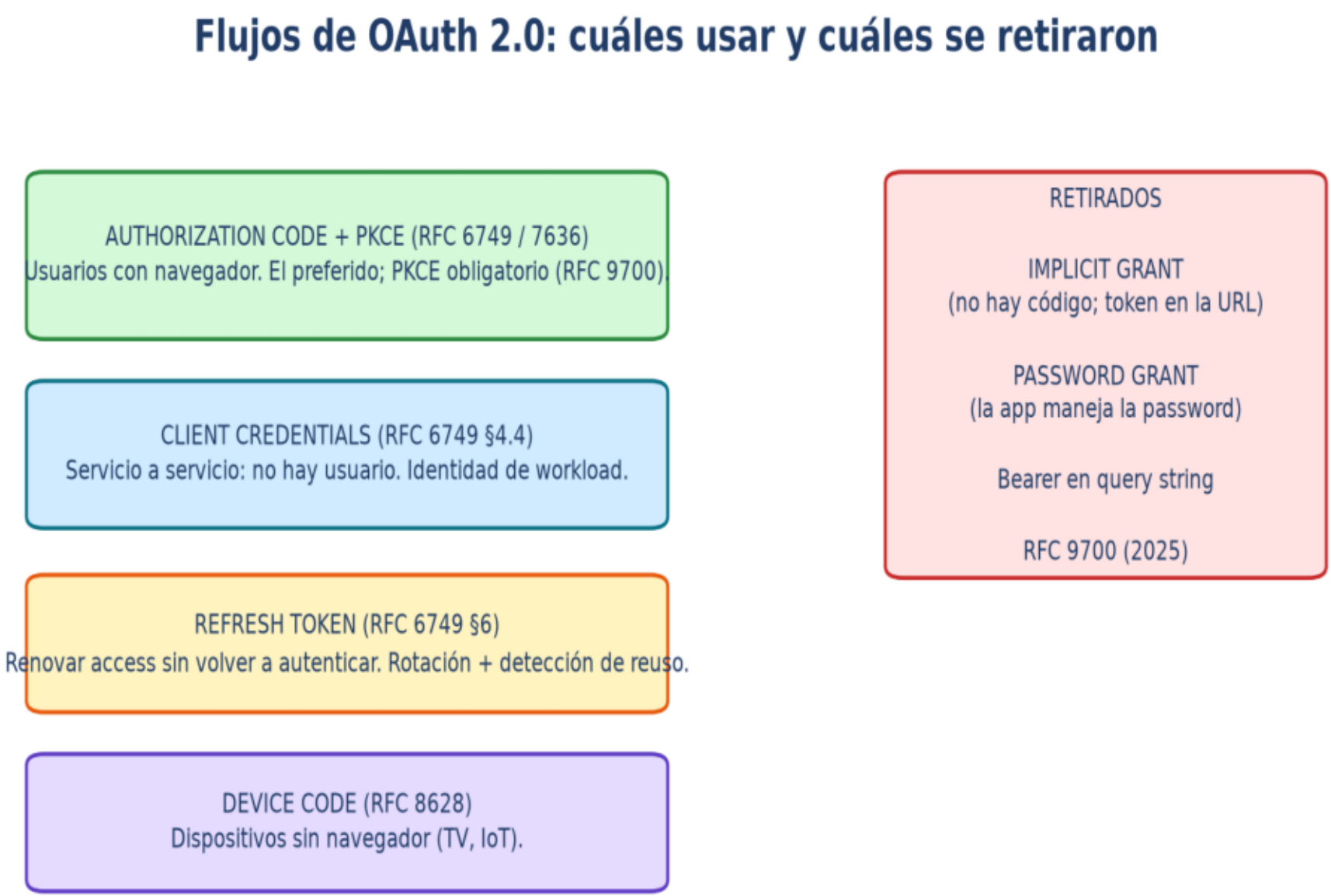


7. Del bearer a los tokens sender-constrained

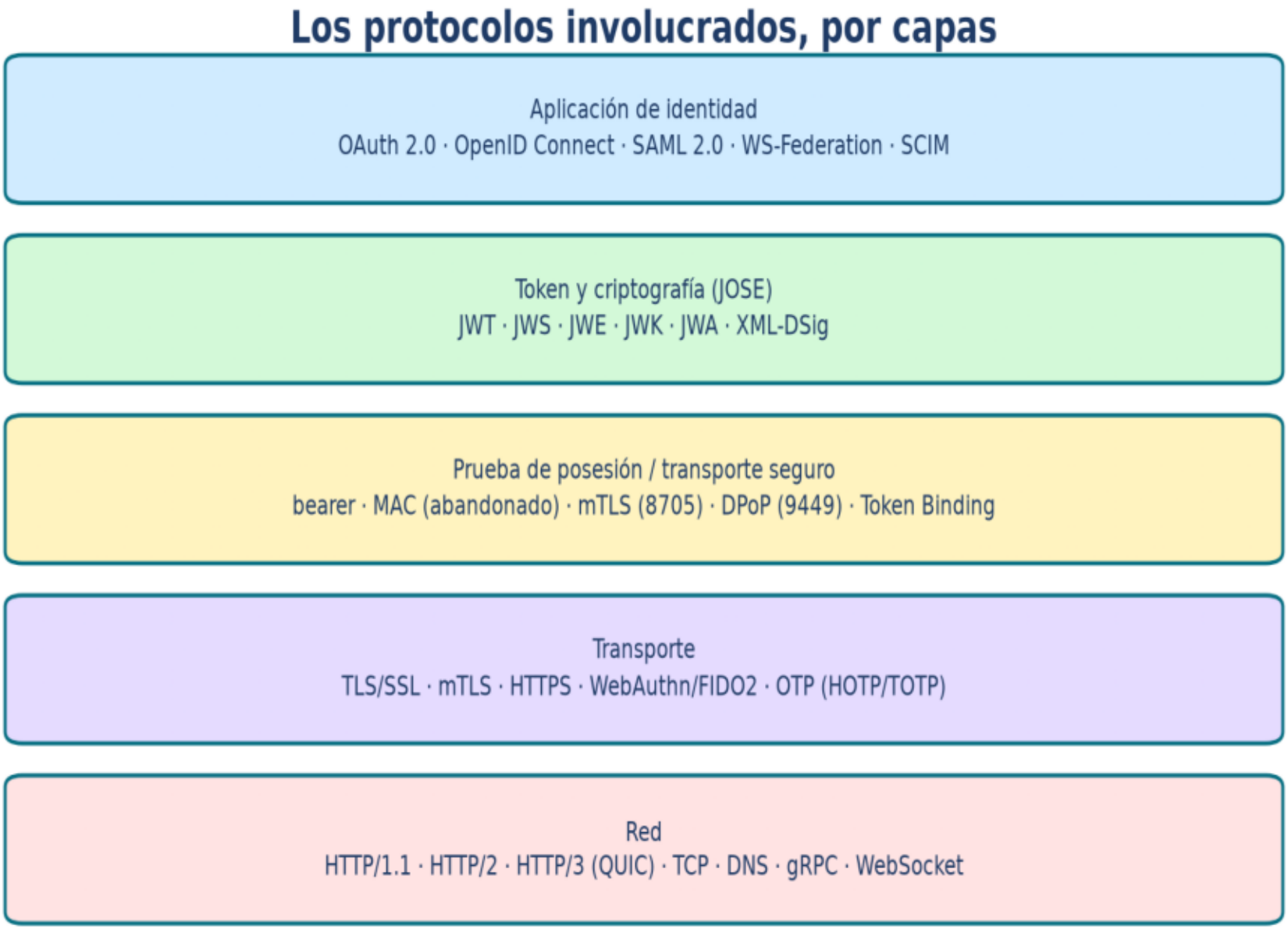


RFC 9700 (2025): los tokens sender-constrained se recomiendan donde sea posible.

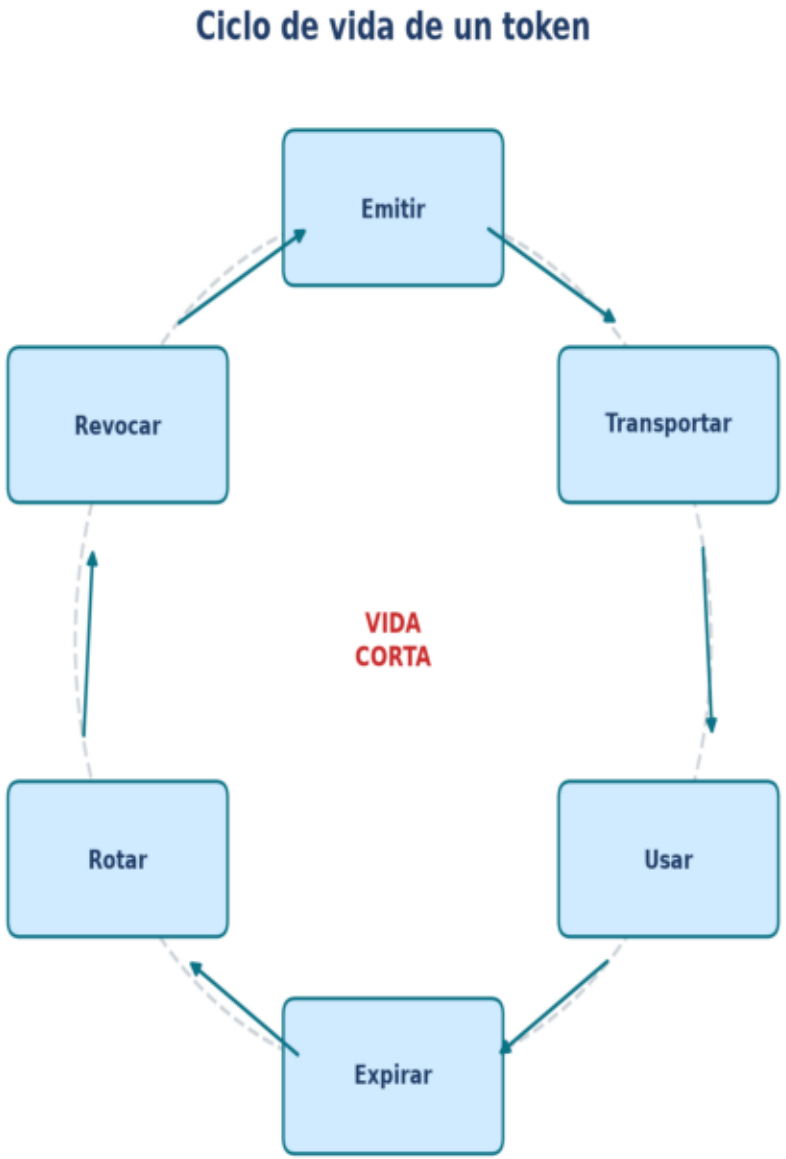
8. Flujos de OAuth 2.0: cuáles usar y cuáles se retiraron



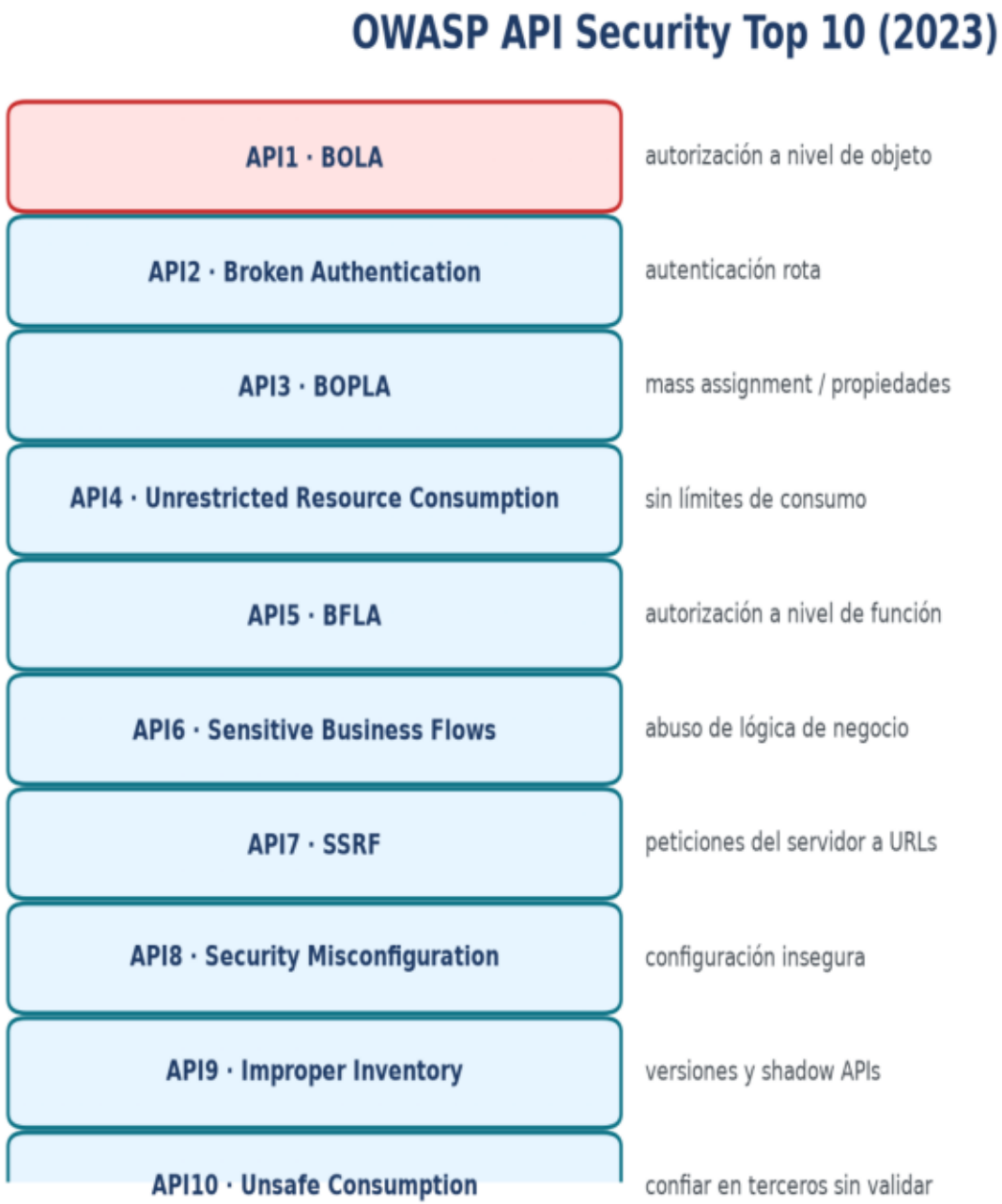
9. Los protocolos involucrados, por capas



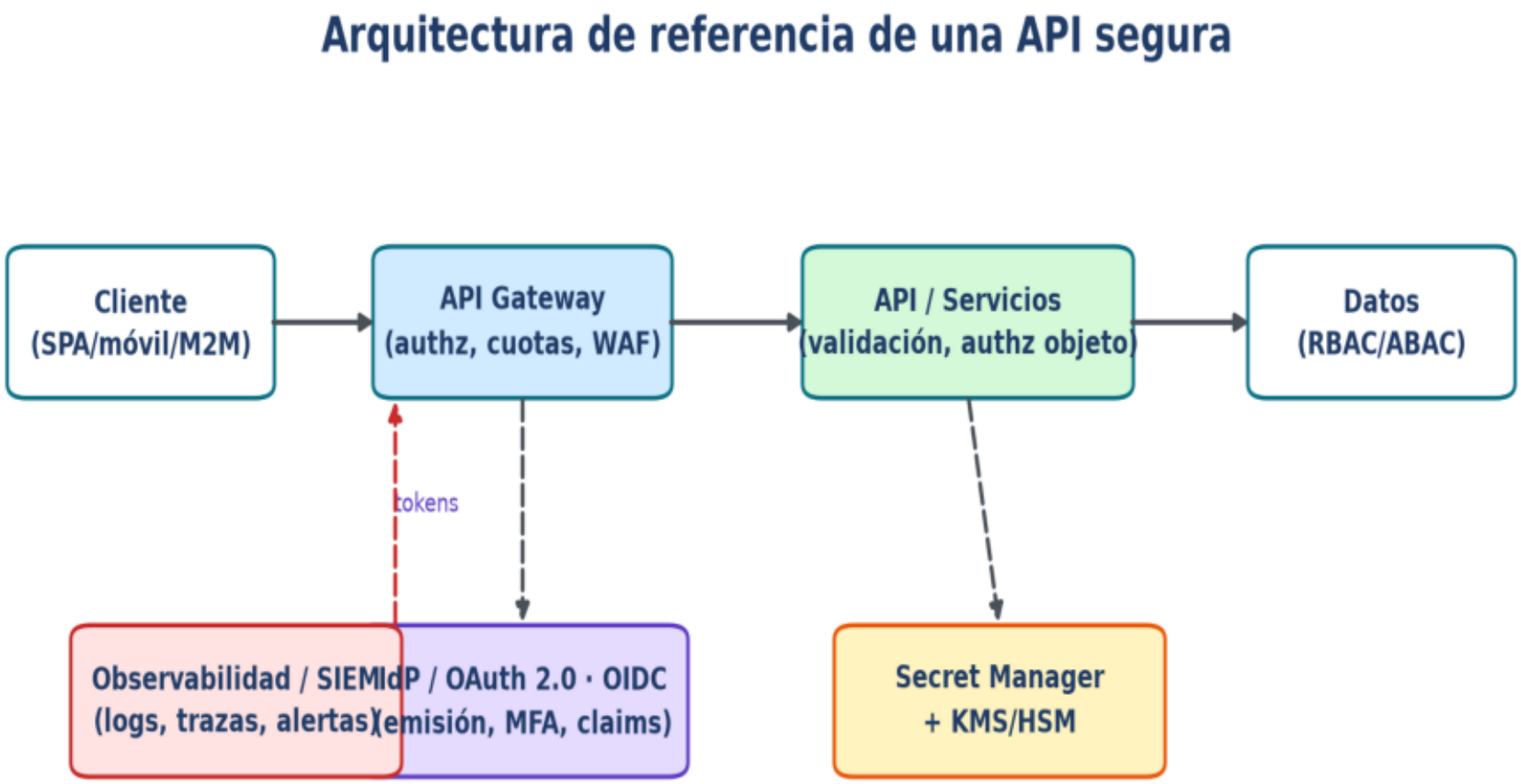
10. Ciclo de vida de un token



11. OWASP API Security Top 10 (2023)



12. Arquitectura de referencia de una API segura



Cada capa agrega un control. El token es solo una de ellas.

13. Alrededor del token hay un sistema entero

